Correctness of the layered pebble game for (K,L)-sparsity

Signe Lundqvist, Tovohery Randrianarisoa, Klara Stokes, Joannes Vermant

arxiv:2306.05050v3; CC BY 4.0

Editorial note. The two displayed corollaries together form one counted algorithm-recognition claim. Their local invariants and reachability lemma are part of the solution obligations, not free background.

Editorial note. The sentence at source905 explicitly assumes G is sparse for the ensuing converse proof. The extracted reachability lemma must be read with that sentence; its isolated statement916 omits that premise.

Editorial note. In source871-872 the occurrence o(e) in v is a typographical variable mismatch in definitions indexed by V-prime. Source868-873 and the conservation identity888 specify membership in the subset V-prime. Original lines are preserved.

Editorial note. The displayed running-time bound942-945 and the independent greedoid theorem are not counted or included.

Editorial scope. One algorithm correctness task combining the author soundness and completeness corollaries, with the complete original local invariant and reachability proof chain. Standard finite graph search is allowed. The running time, greedoid structure theorem, and previously published surplus-independence lemma are outside the counted task.

Allowed standard prerequisites (editorial).

- Finite directed-graph reachability and depth-first search. Reach(v) is the set of vertices accessible by directed paths, and can be found by depth-first search.. Find a path bringing one pebble to an endpoint; no sparsity or matroid theorem is assumed.

Definition 1. *Let $G = (V, E)$ be a (multi)graph such that V has a partition into subsets $V_1, V_2, \dots, V_n$, where every edge $e \in E$ has endpoints in V_i and V_{i+1} for some $i \in \{1, \dots, n-1\}$.*

Let $K = (k_1, \dots, k_n) \in \mathbb{R}^n_{>0}$ and $L = (l_{1,2}, \dots, l_{n-1,n}) \in \mathbb{R}^{n-1}_{\geq 0}$. For any $V' \subseteq V$, we define

$$l(V') = \sup_{\substack{i: V_i \cap V' \neq \emptyset \\ V_{i+1} \cap V' \neq \emptyset}} \{l_{i,i+1}\}.$$

We say that G is (K, L)-sparse if, for any subset $V' \subseteq V$, one has

$$|E(V')| \leq \sum_{i=1}^{n} k_i |V' \cap V_i| - l(V').$$

We note that the use of sup *makes* $l(V') = -\infty$ *whenever the defining set is empty.*
Furthermore, G *is* (K, L)*-tight if* G *is* (K, L)*-sparse, and additionally*

$$|E| = \sum_{i=1}^{n} k_i |V \cap V_i| - \sup_{i=1,\ldots,n-1} \{l_{i,i+1}\}.$$

We furthermore say that a subset of edges $E' \subseteq E$ *is* (K, L)*-sparse or* (K, L)*-tight if the graph* (V, E') *is* (K, L)*-sparse or* (K, L)*-tight, respectively. We say that a subset of edges* $E' \subseteq E$ *is maximally sparse if* E' *is* (K, L)*-sparse, and if for any edge* $vw \in E \setminus E'$ *such that* $v, w \in V(E')$, $E' \cup \{vw\}$ *is not* (K, L)*-sparse.*

We will mainly consider the case where the entries of K and L are integers, and we need the following inequalities to hold for the (K, L)-sparse graphs to be a family of graphs that is well-behaved.

$$k_i + k_{i+1} > l_{i,i+1}, \tag{1}$$
$$k_{i+1} + l_{i-1,i} > l_{i,i+1}, \tag{2}$$
$$l_{1,2} \leq \ldots \leq l_{n-1,n}. \tag{3}$$

1 Pebble game algorithm

We assume that we are given a (multi)-graph $G = (V, E)$, as in Definition 1, and we want to test for (K, L)-sparsity. To do this, we introduce a pebble game algorithm.

During the pebble game, we maintain the following data

- A directed multigraph D, with $V(D) = V$, and with edge set being a subset of E. We call the edges of D *accepted* edges. For each edge $e \in D$, directed from v to w, we will use the notation $o(e) := v$ and $t(e) := w$.
- For each vertex $v \in V(D)$, we store a natural number $\mathrm{peb}(v)$, representing the number of 'pebbles' on the node v.

The following two actions are allowed to construct the graph. They are the key steps that will be performed in the algorithm.

- **Accept-Edge**(e): If $e = vw$ is an edge in $E(V_i \cup V_{i+1})$ that is not in D, and $\mathrm{peb}(v) + \mathrm{peb}(w) > l_{i,i+1}$, either remove a pebble from v (if $\mathrm{peb}(v) > 0$), add (v, w) to D, and orient the edge from v to w, or remove a pebble from w (if $\mathrm{peb}(w) > 0$), add (w, v) to D, and orient the edge from w to v.
- **Move-Pebble**$(v \leftarrow w)$: A depth-first search algorithm is done to search for a directed path whose source is v and whose endpoint w contains a pebble. If there exists a directed path $vf_1 \ldots f_k w$ of arcs f_i in D from v to w, and if $\mathrm{peb}(w) > 0$, add a pebble to v, remove a pebble from w and reverse the orientation for every arc f_i.

We can now describe the pebble game algorithm in pseudocode in Algorithm 1. We handle the edges in a determined order. The idea is to first process the edges between V_1 and V_2, then process the edges between V_2 and V_3, and so on. Within the sets $E(V_i \cup V_{i+1})$, for $i \in \{1, \ldots, n-1\}$, any order is allowed.

Algorithm 1 Pebble game for (K, L)-sparse graphs

Input: A graph G, with a partition $V = V_1 \cup \cdots \cup V_n$, such that any $e \in E$ has endpoints in V_i and V_{i+1} for some $1 \leq i \leq n-1$.
Parameters $K \in \mathbb{Z}^n_{>0}, L \in \mathbb{Z}^{n-1}_{\geq 0}$, which satisfy inequalities (1), (2) and (3).
Output: (K, L)-sparsity and/or (K, L)-tightness of G

For each vertex, set $\mathrm{peb}(v) \leftarrow k_i$ if $v \in V_i$ *(Initialise number of pebbles).*
$D \leftarrow (V, \emptyset)$ *(Initialise D).*
for $i \in \{1, \cdots, n-1\}$ **do**
  for $e = vw \in E$ with $v \in V_i$ and $w \in V_{i+1}$ **do**
    while e has not been processed **do**
      if $\mathrm{peb}(v) + \mathrm{peb}(w) > l_{i,i+1}$ **then**
        Accept-Edge(e).
        e has been processed.
      else
        Depth first search for a directed path $(v' f_1 \dots f_n u)$, with $f_j \in E(D)$, $v' \in \{v, w\}$, where $u \notin \{v, w\}$, such that $\mathrm{peb}(u) > 0$
        if $(v' \dots u)$ with $\mathrm{peb}(u) > 0$ has been found **then**
          Move-Pebble$(v' \leftarrow u)$.
        else
          return G is not sparse.
        end if
      end if
    end while
  end for
end for
if $\sum_{v \in V} \mathrm{peb}(v) = \max L$ **then**
  return G is tight.
else
  return G is sparse, but not tight.
end if

Corollary 2. *Let $K \in \mathbb{Z}^n_{>0}, L \in \mathbb{Z}^{n-1}_{\geq 0}$ be such that inequalities* (1), (2) *and* (3) *are valid. Let $G = (V, E)$ be a graph such that the vertex set has a partition $V = V_1 \cup \cdots \cup V_n$ such that all edges $e \in E$ have endpoints in $V_i \cup V_{i+1}$ for some $1 \leq i < n$.*

If all edges of G are accepted in the pebble game, then G is (K, L)-sparse. If there are exactly $l_{n-1,n}$ pebbles left in the graph, then G is tight.

Corollary 3. *Let $K \in \mathbb{Z}^n_{>0}, L \in \mathbb{Z}^{n-1}_{\geq 0}$ be such that inequalities* (1), (2) *and* (3) *are valid. Let $G = (V, E)$ be a graph such that the vertex set has a partition $V = V_1 \cup \cdots \cup V_n$ such that all edges $e \in E$ have endpoints in $V_i \cup V_{i+1}$ for some $1 \leq i < n$.*

If the graph G is (K, L)-sparse, then the pebble game accepts all edges of G.

Let us now prove that this pebble game recognises the (K, L)-sparse graphs that we have defined. For every subset $V' \subseteq V$, we define the following quanti-

ties.

$$\begin{aligned}\mathrm{peb}(V') &:= \sum_{v\in V'} \mathrm{peb}(v),\\ \mathrm{span}(V') &:= \#\{e \in D \mid o(e) \in v \text{ and } t(e) \in V'\},\\ \mathrm{out}(V') &:= \#\{e \in D \mid o(e) \in v \text{ and } t(e) \notin V'\}.\end{aligned}$$

Lemma 4. *Let $K \in \mathbb{Z}^n_{>0}, L \in \mathbb{Z}^{n-1}_{\geq 0}$ be such that inequalities* (1), (2) *and* (3) *are valid. Let $G = (V, E)$ be a graph such that the vertex set has a partition $V = V_1 \cup \cdots \cup V_n$ such that all edges $e \in E$ have endpoints in $V_i \cup V_{i+1}$ for some $1 \leq i < n$. The following are invariants of the algorithm*

1. *For all $v \in V_i$, one has $\mathrm{peb}(v) + \mathrm{out}(v) = k_i$.*
2. *Suppose that $V' \subseteq V$. Then*

$$\mathrm{peb}(V') + \mathrm{span}(V') + \mathrm{out}(V') = \sum_{i=1}^{n} k_i |V' \cap V_i|.$$

3. *Consider a nonempty subset $V' \subseteq V$. Then*

$$\mathrm{peb}(V') + \mathrm{out}(V') \geq l(V').$$

4. *Consider a subset $V' \subseteq V$. Then*

$$\mathrm{span}(V') \leq \sum_{i=1}^{n} k_i |V' \cap V_i| - l(V').$$

Proof. The technique of the proof is similar to that of Lemma 10 in [1], and we only briefly sketch the proofs of invariants 1, 2, and 4 for completeness. Since invariant 3 requires more analysis, we give the proof in more detail. For the first point, this holds at the start of the algorithm. By considering what happens when moving a pebble or accepting an edge, one sees that the invariant remains valid. For invariant 2, we see that

$$\sum_{v\in V'} \mathrm{peb}(v) + \mathrm{out}(v) = \mathrm{peb}(V') + \mathrm{span}(V') + \mathrm{out}(V') = \sum_{i=1}^{n} k_i |V' \cap V_i|.$$

We now show that the third invariant holds after every step in the algorithm. Take a subset $V' \subseteq V$. By the convention for the maximum defining $l(V')$, if there is no i with $V_i \neq \emptyset$ and $V_{i+1} \neq \emptyset$, then $l(V') = -\infty$, and there is nothing to show. So, we assume that $l(V') = l_{i,i+1}$ for some $i \in \{1, \cdots, n-1\}$.
Suppose that an edge has been accepted with endpoints in V' in the previous step of the algorithm. We need to distinguish between various cases to show the inequality. If the edge is between vertices x, y with $x \in V_i$, and $y \in V_{i+1}$, then there must be $l_{i,i+1}$ pebbles left after accepting, since there need to have been at least $l_{i,i+1} + 1$ pebbles before the edge was accepted. If the accepted edge was between vertices x, y with $x \in V_{i-1}$, and $y \in V_i$, then there must be at least $l_{i-1,i}$ pebbles at x and y, and there must be a vertex z with $\mathrm{peb}(z) = k_{i+1}$, since the edges in $E(V_i \cup V_{i+1})$ are processed after those in $E(V_{i-1} \cup V_i)$. Thus

there must be $l_{i-1,i} + k_{i+1} > l_{i,i+1}$ pebbles left in V', by inequality (2). If there was an edge accepted between vertices x, y with $x \in V_k$ and $y \in V_{k+1}$, with $k < i-1$, then there must be vertices z and w (by the order in which edges are accepted) with $\text{peb}(z) = k_i$, $\text{peb}(w) = k_{i+1}$, and thus there are $k_i + k_{i+1} \geq l_{i,i+1}$ pebbles left. If an edge gets accepted outside of V', or if a pebble gets moved, $\text{peb}(V') + \text{out}(V')$ does not change, which can be seen by a case analysis.

The fourth point follows immediately from the third and second invariant, by subtracting the third invariant from the second one. □

Proof. This is an easy consequence of the fourth and second invariants of Lemma 4. □

We have now shown that if every edge in G is accepted in the pebble game, then G is sparse. Now we show the converse, namely that if G is sparse, then every edge of G will be accepted.
In the following proof, we need to define the notion of the *Reach* of a vertex in an oriented graph D. Let

$$\text{Reach}(v) := \{v\} \cup \{w \in V(D) \mid \text{ there exists an oriented path from } v \text{ to } w\}.$$

Lemma 5. *Let $K \in \mathbb{Z}^n_{>0}, L \in \mathbb{Z}^{n-1}_{\geq 0}$ be such that inequalities* (1)*,* (2) *and* (3) *are valid. Let $G = (V, E)$ be a graph such that the vertex set has a partition $V = V_1 \cup \cdots \cup V_n$ such that all edges $e \in E$ have endpoints in $V_i \cup V_{i+1}$ for some $1 \leq i < n$.*

Suppose that after running some steps of the algorithm D is the set of edges we accepted, where all edges in D are between V_a and V_{a+1} for $a \leq i$. Suppose that e is an edge of G which has not been accepted yet such that $e = (uv)$, where $u \in V_i$ and $v \in V_{i+1}$. Then the edge can be accepted (after perhaps moving some pebbles).

Proof. If there are more than $l_{i,i+1}$ pebbles at u and v together, we can add the edge and remove a pebble. So, we may assume that $\text{peb}(u) + \text{peb}(v) < l_{i,i+1} + 1$. Let $V' = \text{Reach}(u) \cup \text{Reach}(v)$, and let $E_G(V')$ denote the edges induced by V' in G. Note that $V' \cap V_i$ and $V' \cap V_{i+1}$ are non-empty since $u \in V' \cap V_i$ and $v \in V' \cap V_{i+1}$. Furthermore $V' \cap V_k = \emptyset$, for $k \geq i+2$, since these edges get processed at some later point. Since G is sparse,

$$\text{span}(V') < |E_G(V')| \leq \sum_{j=1}^{i+1} k_j |V' \cap V_j| - l_{i,i+1},$$

where the first inequality is strict since $e \in E_G(V') \backslash \text{span}(V')$, and $\max\{l_{k,k+1}\} = l_{i,i+1}$ since the $l_{k,k+1}$ are increasing. By the definition of Reach, $\text{out}(V') = 0$ and by invariant 2 in Lemma 4,

$$\text{span}(V') + \text{peb}(V') = \sum_{j=1}^{i+1} k_j |V_j \cap V'|$$

and thus

$$\text{peb}(V') > l_{i,i+1}.$$

Hence, there is a vertex $w \in V'$ such that $w \notin \{v, u\}$ with at least one pebble left. Since it is reachable by either v or u, we can move that pebble (changing

the direction of the path). Repeat this process until we get enough pebbles together at u and v so that we can accept the edge (uv) and add it to D. □

Proof. This follows easily from Lemma 5. □

References

[1] Audrey Lee and Ileana Streinu. Pebble game algorithms and sparse graphs. *Discrete Mathematics*, 308(8):1425–1437, 2008. Third European Conference on Combinatorics.